

Contents

▼

SETUP, START TO FINISH

Installation guide

Everything needed to get FreightFast running from a fresh copy of the source — requirements, setup steps, configuration, and the fixes for the handful of things that most commonly trip people up. For what the product actually does, see the [Documentation](#).

01 — BEFORE YOU START

Requirements

COMPONENT	VERSION	
Node.js	20.x or newer	required
npm	10.x or newer	required
MySQL or MariaDB	8.0 / 10.6+	required
Git	any recent	recommended
Azure Blob Storage account	—	optional
SMTP / WhatsApp Cloud API / SMS gateway	—	optional
Google AI Studio API key	for Gemini	optional

Document uploads, SMTP/WhatsApp/SMS delivery, and AI features all work without their optional credentials configured — the app runs with those subsystems simply switched off until you supply keys.

02 — SETUP

Installation

Seven steps from a fresh copy of the source to a running app with demo data loaded.

1

Install dependencies

From the project root:

```
npm install
```

2

Create the database

Create an empty MySQL/MariaDB database for the app. Using the MySQL CLI:

```
mysql -u root -p -e "CREATE DATABASE freight_control CHARACTER SET utf8mb4;"
```

3

Configure environment variables

Copy the example file and fill in your own values:

```
cp .env.example .env
```

At minimum, set `DATABASE_URL` to point at the database from step 2, and generate a random `NEXTAUTH_SECRET`. See [Environment variables](#) for the full list.

```
# macOS / Linux
openssl rand -hex 32

# Windows PowerShell
[Convert]::ToHexString((1..32 | % { Get-Random -Max 256 } ))
```

4

Generate the Prisma client

```
npx prisma generate
```

5

Push the schema

Creates every table in the database from `prisma/schema.prisma`:

```
npx prisma db push
```

6

Seed initial data

Creates the platform owner, a demo company, its default roles, and a set of demo staff accounts:

```
npx prisma db seed
```

Credentials for every account it creates are listed under [Default accounts](#).

7

Start the app

For local development, with hot reload:

```
npm run dev
```

Then open `http://localhost:3000`. If port 3000 is already in use, Next.js picks the next free port automatically and prints it in the terminal — watch the startup output for the actual URL.

i Upgrading an existing install? Re-run `npx prisma db push` (or apply the migrations under `prisma/migrations/` in order) after pulling new code, then restart the app server. New tables and columns are additive and nullable by default, so existing data keeps working unchanged.

03 — CONFIGURATION REFERENCE

Environment variables

Grouped by subsystem. Anything marked optional can be left blank — that feature stays switched off rather than erroring.

Database & auth

VARIABLE	PURPOSE	
DATABASE_URL	MySQL connection string, e.g. <code>mysql://user:pass@localhost:3306/freight_control</code>	required
NEXTAUTH_URL	Public base URL of the app, e.g. <code>http://localhost:3000</code>	required
NEXTAUTH_SECRET	Random secret used to sign session tokens. Changing it invalidates all existing sessions.	required
ADMIN_APP_HOST	Hostname routed to the platform console. Defaults to a placeholder if unset.	optional
COMPANY_APP_HOST	Hostname routed to the company dashboard.	optional
PORTAL_APP_HOST	Hostname routed to the client portal.	optional

Seed data

VARIABLE	PURPOSE	
SEED_ADMIN_NAME	Display name for the seeded company admin.	optional
SEED_ADMIN_EMAIL	Login email for the seeded company admin. Falls back to <code>admin@freightcontrol.com</code> .	optional
SEED_ADMIN_PASSWORD	Password for every seeded staff account. Falls back to <code>Admin123</code> .	optional

File storage		
VARIABLE	PURPOSE	
AZURE_STORAGE_CONNECTION_STRING	Connection string from Azure Portal → Storage Account → Access keys.	optional
AZURE_STORAGE_CONTAINER_NAME	Blob container for uploaded documents.	optional

Delivery channels — email, WhatsApp, SMS		
VARIABLE	PURPOSE	
EMAIL_DELIVERY_ENABLED	Master switch for outbound email.	optional
SMTP_HOST / PORT / USER / PASS	Your SMTP relay credentials.	optional
SMTP_FROM_EMAIL / SMTP_FROM_NAME / SMTP_SECURE	From-address, display name, and TLS mode.	optional
WHATSAPP_DELIVERY_ENABLED	Master switch for WhatsApp delivery.	optional
WHATSAPP_CLOUD_API_TOKEN / PHONE_NUMBER_ID	Meta WhatsApp Cloud API credentials.	optional
SMS_DELIVERY_ENABLED / PROVIDER / API_KEY / SENDER_ID	SMS gateway configuration.	optional
DELIVERY_DRY_RUN	When true, logs deliveries instead of sending them. On by default.	optional
COMMUNICATION_SECRET_KEY	Encrypts stored provider credentials. Required before any real (non-dry-run) send.	optional

Background job endpoints		
VARIABLE	PURPOSE	
NOTIFICATION_DISPATCH_SECRET	Bearer secret for the notification cron endpoint. Endpoint stays closed until set.	optional
RECURRING_TASK_DISPATCH_SECRET	Bearer secret for the recurring-task cron endpoint. Endpoint stays closed until set.	optional

AI features		
VARIABLE	PURPOSE	

AI_ENABLED	Platform-wide kill-switch. Must be true for any company to use AI tools.	optional
AI_PROVIDER	Currently GEMINI — the only wired provider.	optional
GOOGLE_GENERATIVE_AI_API_KEY	Platform-level Gemini key, used when a company hasn't saved its own.	optional
AI_DEFAULT_DAILY_REQUEST_CAP	Default per-company daily request quota.	optional
AI_SECRET_KEY	Encrypts a company's own saved Gemini API key.	optional

04 — AFTER SEEDING

Default accounts

Running `npx prisma db seed` creates one platform-level account and six accounts inside a demo company. Every password below is the value of `SEED_ADMIN_PASSWORD` — Admin123 if you left it unset.

Platform console — /platform-login

Platform Owner		
email	platform@freightcontrol.com	
password	SEED_ADMIN_PASSWORD	

Company dashboard — /login

Company Admin			Operations Manager			Documentation Officer		
email	SEED_ADMIN_EMAIL		email	operations@freightcontrol.com		email	documentation@freightcontrol.com	
password	SEED_ADMIN_PASSWORD		password	SEED_ADMIN_PASSWORD		password	SEED_ADMIN_PASSWORD	

Accounts Officer			Sales Executive		
email	accounts@freightcontrol.com		email	sales@freightcontrol.com	
password	SEED_ADMIN_PASSWORD		password	SEED_ADMIN_PASSWORD	

Client portal — /portal/demo-freight/login

A demo client account is seeded with a masked client code and a forced password change on first login. Retrieve its current login link and code from Dashboard → Customers → (demo customer) → Portal Access after seeding.

 Rotate every seeded password before exposing an installation beyond your own machine. These accounts exist to get you into the app quickly, not for production use as-is.

Background jobs

Two features rely on an external scheduler hitting a protected endpoint. Both stay inert — the app works fine without them — until you set their secret.

Notification dispatch

Sends approved, queued email/WhatsApp/SMS notifications. In-app notifications work regardless; this only governs *external* delivery.

```
* / 5 * * * * curl -fsS -X POST \
  -H "x-dispatch-secret: $NOTIFICATION_DISPATCH_SECRET" \
  https://your-domain.example/api/cron/notification-dispatch
```

Failed sends back off at 5min → 20min → 1hr → 4hr, up to five attempts, before landing in the Delivery Outbox for manual retry.

Recurring tasks

Generates due occurrences from recurring task templates. An hourly interval covers daily/weekly/monthly rules comfortably.

```
0 * * * * curl -fsS -X POST \
  -H "x-dispatch-secret: $RECURRING_TASK_DISPATCH_SECRET" \
  https://your-domain.example/api/cron/recurring-tasks
```

Without the cron configured, individual recurrences can still be caught up on demand from their detail page using **Run now**.

Production deployment

A production build is a standard Next.js build — point it at a real database and a persistent process manager.

```
npm run build
npm run start
```

Before switching real users onto it: generate a fresh `NEXTAUTH_SECRET` (don't reuse the one from local development), point `DATABASE_URL` at your production database, run the same `prisma generate` → `db push` sequence against it, and set `NEXTAUTH_URL` to the real public URL.

The repository includes ready-made GitHub Actions workflows for deploying to an Azure Web App or a self-hosted VPS over SSH — both read their credentials from GitHub repository

secrets rather than anything in this codebase, so they need no changes here to use; configure the corresponding secrets in your GitHub repository settings.

Troubleshooting

- Sign-in fails with "JWT_SESSION_ERROR — decryption operation failed"

×

This means a session cookie in your browser was encrypted with a different `NEXTAUTH_SECRET` than the one currently configured — typically because the secret was regenerated after a previous sign-in.

Clear cookies for the app's domain (or open it in a private window) and sign in again.
- MySQL says "Access denied for user"

+
- The dev server starts on a different port than 3000

+
- Prisma throws a validation error mentioning a missing relation

+
- Seeding fails on a brand-new database

+